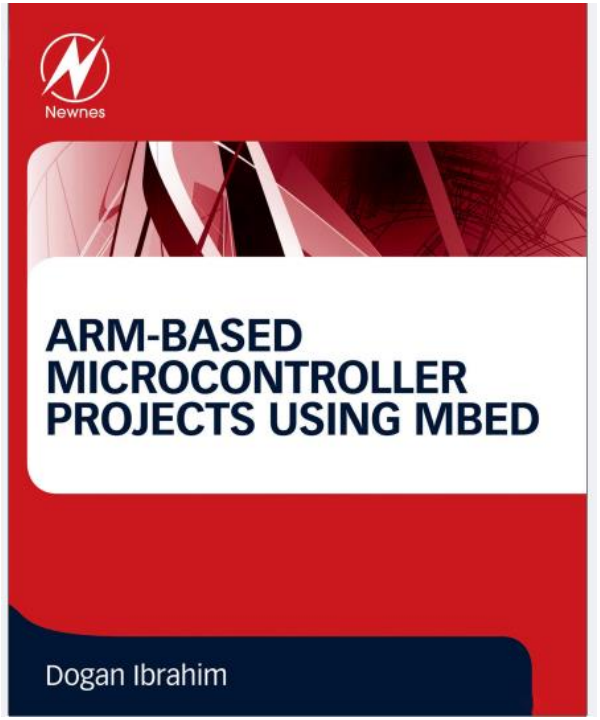
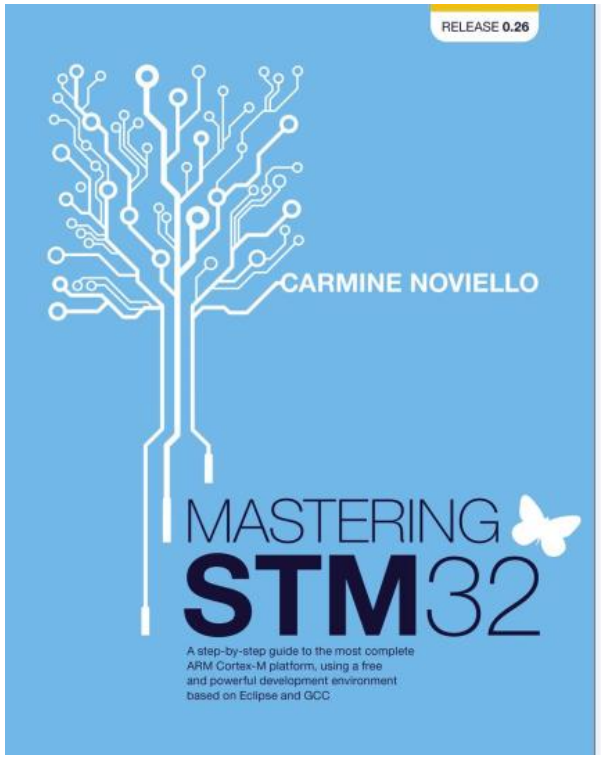
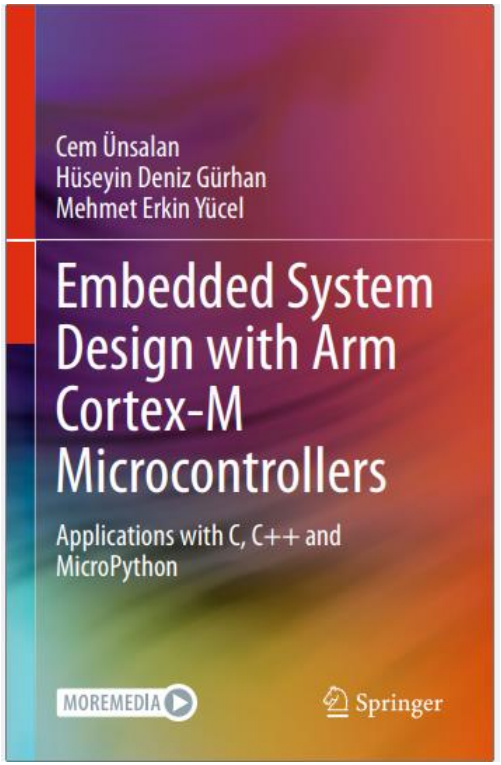
ee2004

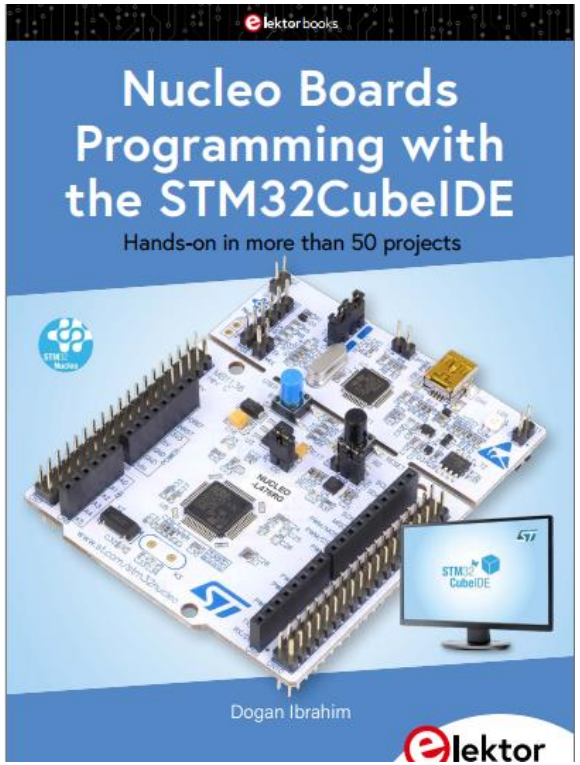
Week1,2

textbooks


Newnes
ARM-BASED MICROCONTROLLER PROJECTS USING MBED
Dogan Ibrahim


RELEASE 0.26
CARMINE NOVIELLO
MASTERING STM32
A step-by-step guide to the most complete ARM Cortex-M platform, using a free and powerful development environment based on Eclipse and GCC


Cem Ünsalan
Hüseyin Deniz Gürhan
Mehmet Erkin Yücel
Embedded System Design with Arm Cortex-M Microcontrollers
Applications with C, C++ and MicroPython
MOREMEDIA
Springer

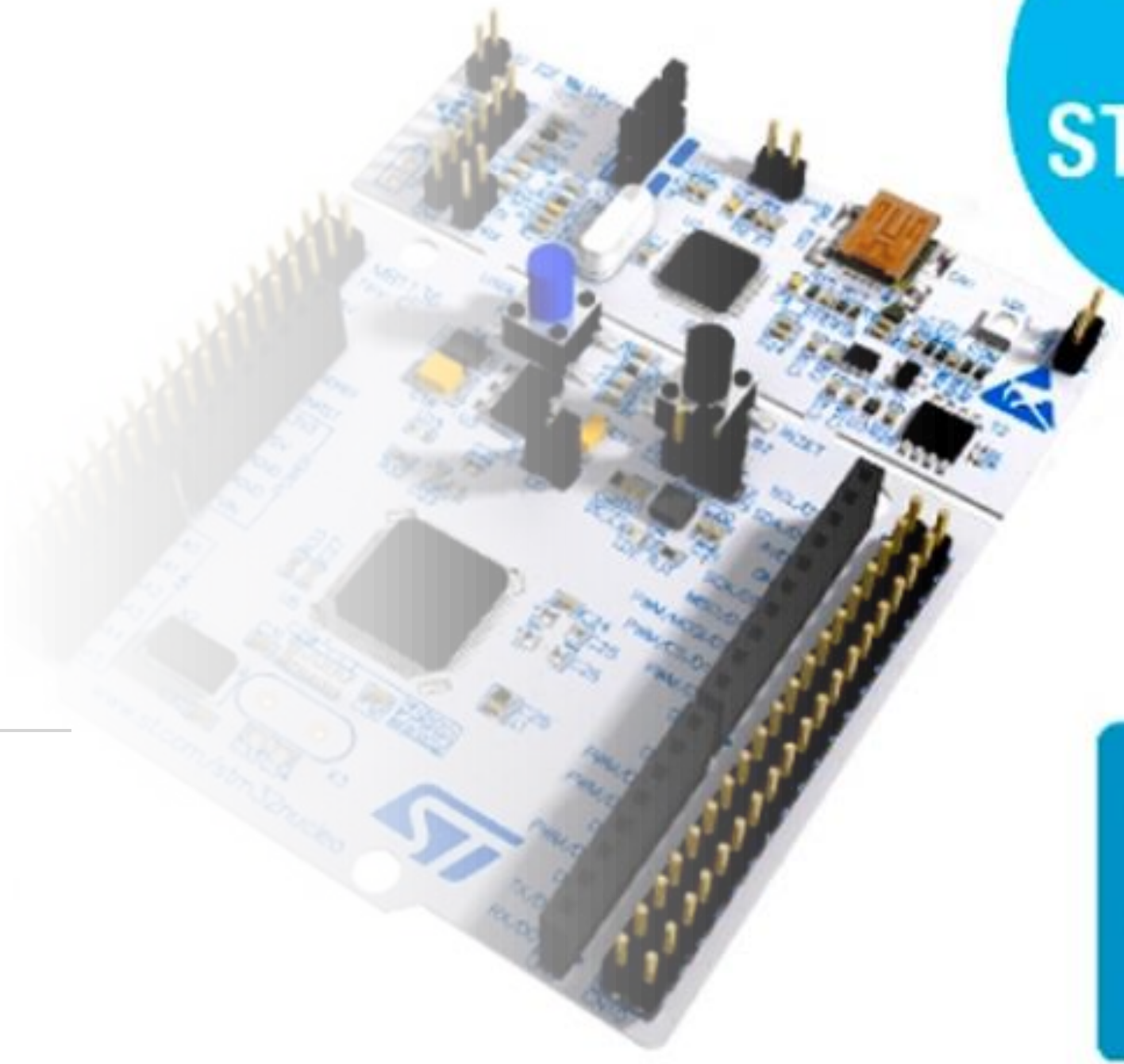

elektor books
Nucleo Boards Programming with the STM32CubeIDE
Hands-on in more than 50 projects
Dogan Ibrahim
elektor

software tools

- VisUAL2: ARM Simulator
- Arm Mbed OS Simulator
- Keil Studio Cloud / Mbed OS
- STM32CubeIDE
- Arduino IDE

board

Stm32 Nucleo



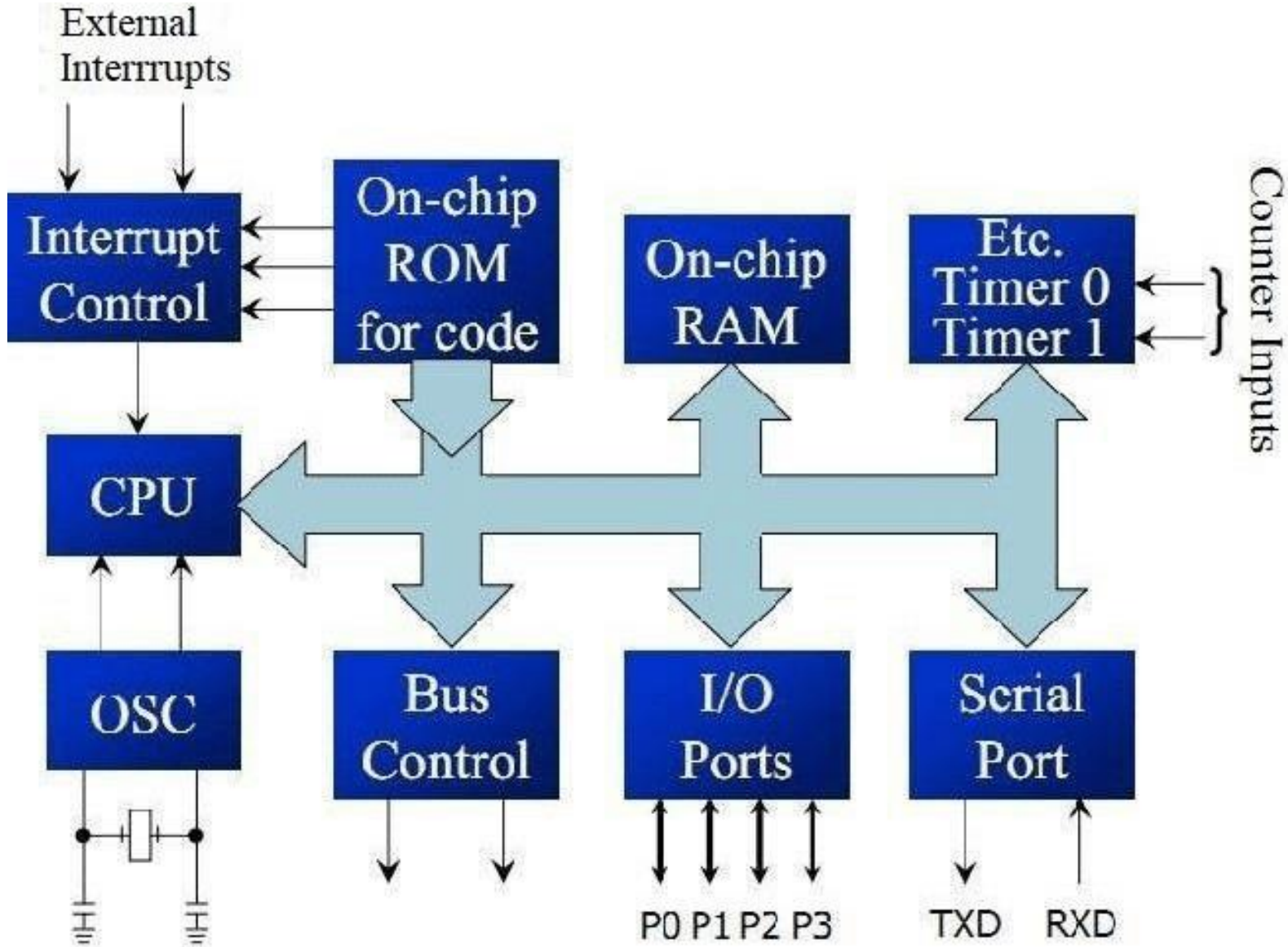
Topics

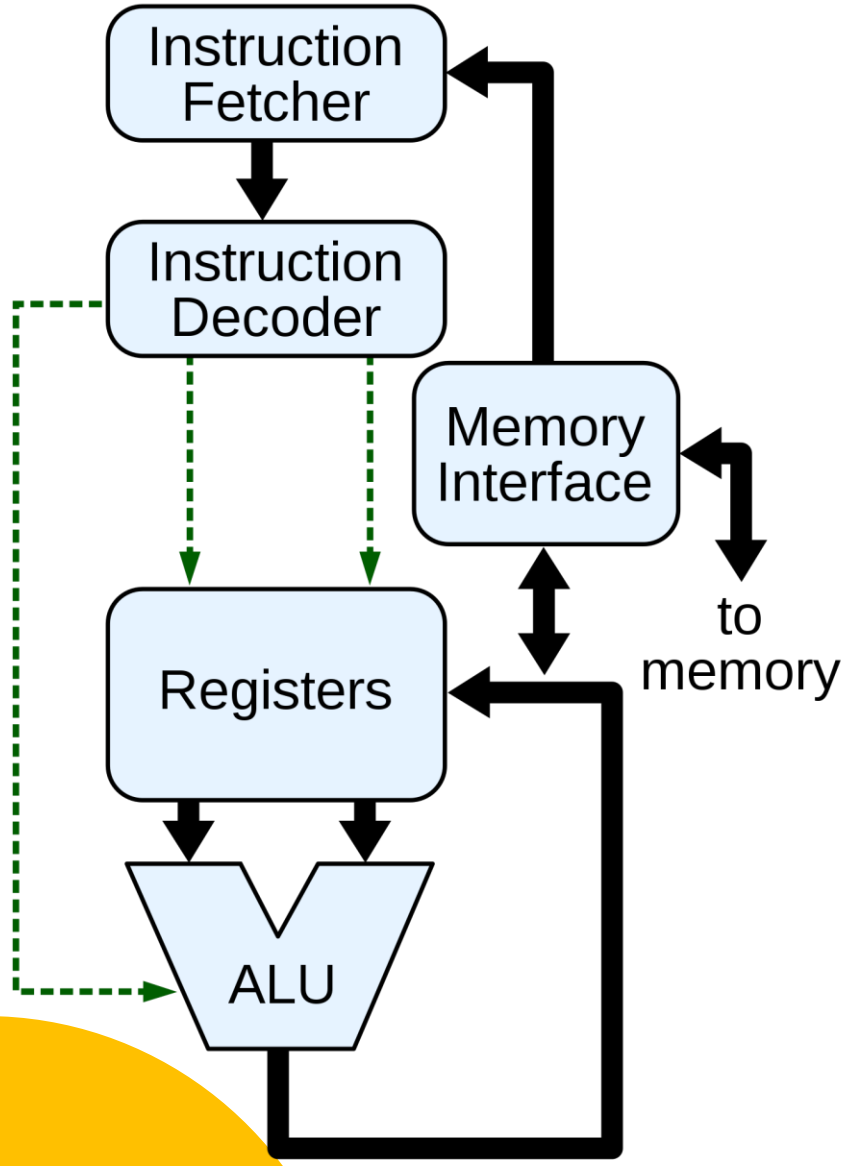
- Introduction
- Architecture
- Assembly Language
- Development Platforms
- Digital Inputs / Outputs
- Interrupts
- Timing Operations
- Data Acquisition
- Digital Communication
- Memory Operations

- An embedded system is a computing device developed for solving a specific problem.
- To do so, it interacts with the environment as acquiring data, processes the acquired data, and produces the corresponding output accordingly.
- To perform all these operations, joint usage of hardware and software becomes mandatory.
- The embedded system will be working in stand-alone form most of the times.
- It may communicate with other nearby devices as with Internet of Things (IoT) applications. Each embedded system can work in stand-alone form and share data to perform a complex operation.
- Since the embedded system works in stand-alone form most of the times, it depends on battery or energy harvesting module to operate.
- Therefore, energy dissipation becomes one of the main concerns in embedded system development.

- We can group embedded systems based on their hardware properties as follows.
- Field-programmable gate arrays (FPGA) form the first group. They provide the most flexible but hard to master hardware. Due to their properties, they have been extensively used in forming custom embedded systems at gate level.
- The second hardware group in forming embedded systems consists of microcontrollers. A microcontroller has limited memory and computation power. However, it can be programmed in assembly or high-level language to perform the operations on it.
- The microprocessors form the third embedded system group. In these, embedded Linux is used most of the times to control and organize operations. The most well known microprocessor-based embedded systems are the Raspberry Pi family and its clones.
- The fourth hardware group for embedded systems consists of system on chip (SoC) devices. They have FPGA and microprocessor modules on them. Hence, they aim to benefit from both device properties.
- We picked the microcontroller as the embedded system in this course.

- Each microcontroller family has its own programming language called assembly.
- The assembly language allows the programmer to reach and modify the microcontroller hardware at its lowest level.
- Although this is a fairly powerful tool, there are some shortcomings as well.
- First and most important of all, an assembly code written for a microcontroller family cannot be ported to a microcontroller from another family.
- Hence, C became the de-facto programming language for microcontrollers.
- Integrated development environments allow the user to control and debug the code running on the microcontroller fairly easily.
- Likewise, Arm® introduced the Mbed development environment for its microcontrollers. This platform depends on C++ language to operate.

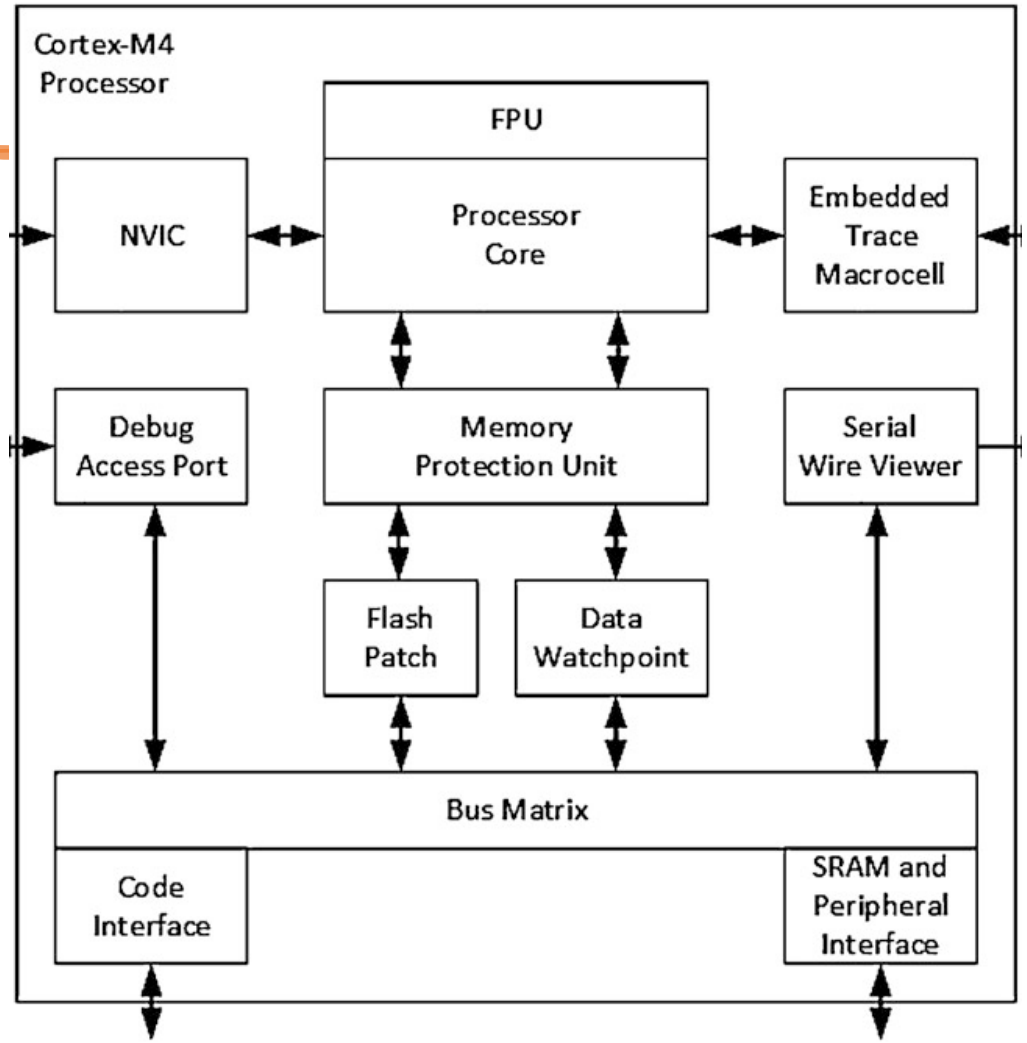

External
Interrrupts
Interrupt
Control
On-chip
ROM
for code
On-chip
RAM
Etc.
Timer 0
Timer 1
Counter Inputs
CPU
OSC
Bus
Control
I/O
Ports
Serial
Port
P0 P1 P2 P3
TXD
RXD



- Grouping hardware and peripheral units of the microcontroller as CPU, memory, general-purpose input and output ports, clock and timer modules, analog modules, digital communication modules, and other modules is beneficial for simplicity.
- Central processing unit (CPU) is responsible for organizing all operations within the microcontroller. This is done by the instructions provided to it by the programmer.
- The programmer constructs algorithm and forms the corresponding C, C++, or assembly code. This code is debugged and embedded into flash memory of the microcontroller by the help of an integrated development environment (IDE, such as STM32CubeIDE) running on PC.
- The CPU executes the commands by using its resources (such as peripheral units).

- CPU of the STM32F4 microcontroller:

- This diagram gives insight about the inner working principles of the CPU as will be explained next.
- Wire connections and their direction are not random in this figure, they indicate the direction of dataflow from and to the CPU.
- The processor core (as indicated in the figure) has an accompanying floating-point unit (FPU). This unit is responsible for handling operations on
g floating-point unit (FPU). This unit is responsible for handling operations on floating-point numbers in hardware. Hence, they can be executed in a fast manner.

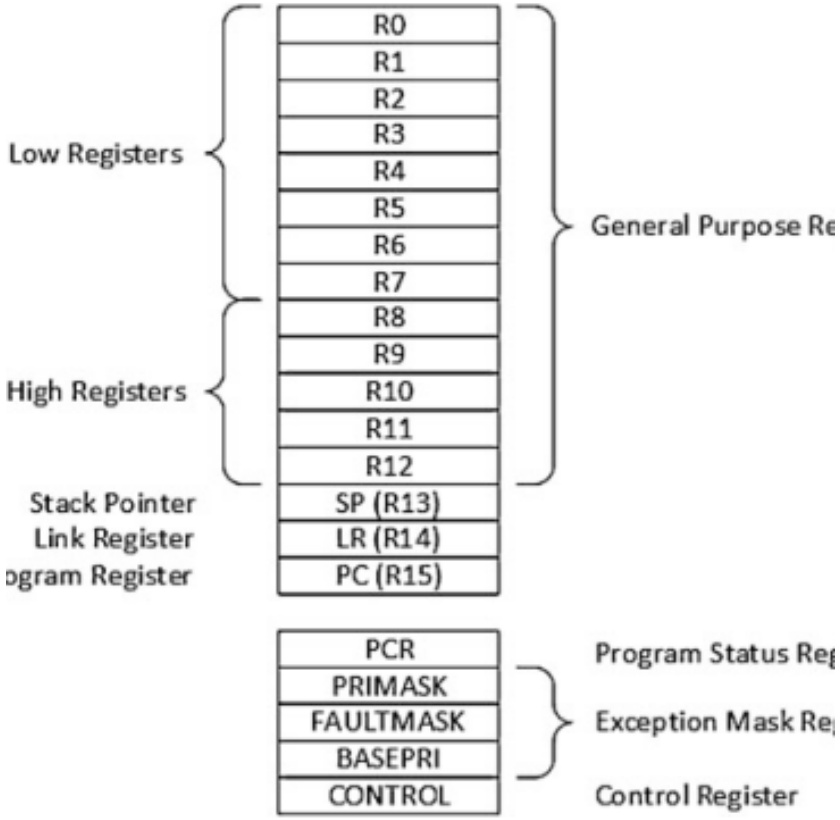


- The nested vectored interrupt controller (NVIC) is responsible for managing interrupt- and exception-based operations within the CPU. We will talk about what an interrupt is and how it is handled by NVIC in detail.
- The debug access port (DAP) helps us to reach and monitor the CPU during operation. Hence, we can observe what is going on inside the CPU while executing code. In other words, DAP allows us to monitor CPU registers without affecting the CPU operation.
- The other option to perform the same operation is leading the CPU to the debug state. Here, the CPU operation is halted. Then, its registers can be monitored.
- There is also the serial wire viewer (SWV) performing a similar operation. Through it, the user can trace the program counter, variable and peripheral register values, exceptions, time stamp, and CPU cycles.
- Data and codes are stored in memory will be explained in detail next week. Memory protection unit, flash patch, and data watchpoints modules within the CPU are used to manage memory.

- The embedded trace macrocell (ETM) module allows monitoring the code execution within the CPU (without stopping it). This is done by the GPIO pins. Therefore, the user can check when the CPU enters an interrupt, which task uses a common variable, and where an error has occurred if there is any. The ETM module needs external modules to perform all these operations.
- There is a bus matrix which connects modules within the CPU. Let's first explain what bus means. In simplest terms, bus is composed of parallel wires in which data (and code blocks) are transferred within the CPU.
- Adjacent to the bus matrix, there are two blocks: the first one being code and the second one being SRAM and peripheral interface.
- The code interface block is responsible for retrieving code blocks to be processed from memory.
- The SRAM and peripheral interface block is important for two reasons. First, an external memory module can be added to the microcontroller through this interface. Second, all peripheral units will be reached as if they are memory addresses. The SRAM and peripheral interface block module will be used for this purpose.

- The microcontroller needs a medium to transfer data to and from the outside world. This is done by using ports of the microcontroller. A port is a collection of pins grouped together. Each pin can be taken as a wire with its electronic control circuitry.
- The CPU and peripheral units within the microcontroller are synchronous devices. This means that operations within them are synchronized by a common clock signal.
- Frequency of the clock signal decides on the speed of operation in actual time. The clock tick idiom indicates the period of the clock signal. The time needed to perform an operation in the microcontroller is indicated by clock pulses (or clock ticks). Therefore, the higher the clock frequency, the faster that operation is performed in actual time. Clock signals are generated by oscillator circuitry which can be located inside or outside the microcontroller.
- The ADC module generates digital representation of an analog voltage level fed to the microcontroller.
- The DAC module converts a digital representation to analog voltage level and feeds it to output.
- Digital communication modules are used to communicate with external devices using dedicated communication protocols.

- Registers are data storage elements on the CPU. The STM32F4 microcontroller has 21 core registers in which 13 of them (R0 to R12) can be used for general purpose. The remaining eight registers are reserved for specific operations by the CPU.
- General-purpose registers (R0–R12) can be used for data processing operations. These are divided into two groups as low register (R0–R7) and high register (R8– R12) depending on their usage by 16- or 32-bit instructions.
- The remaining eight registers are used for specific purposes. Among these, the program counter (R15, PC) is the register responsible for holding address of the instruction to be executed. Related to this, there is a link register (R14, LR) which holds return address during branching and exception operations such as interrupts. Stack pointer (R13, SP) is responsible for storing the location of local variables and is used when a function is called in code.

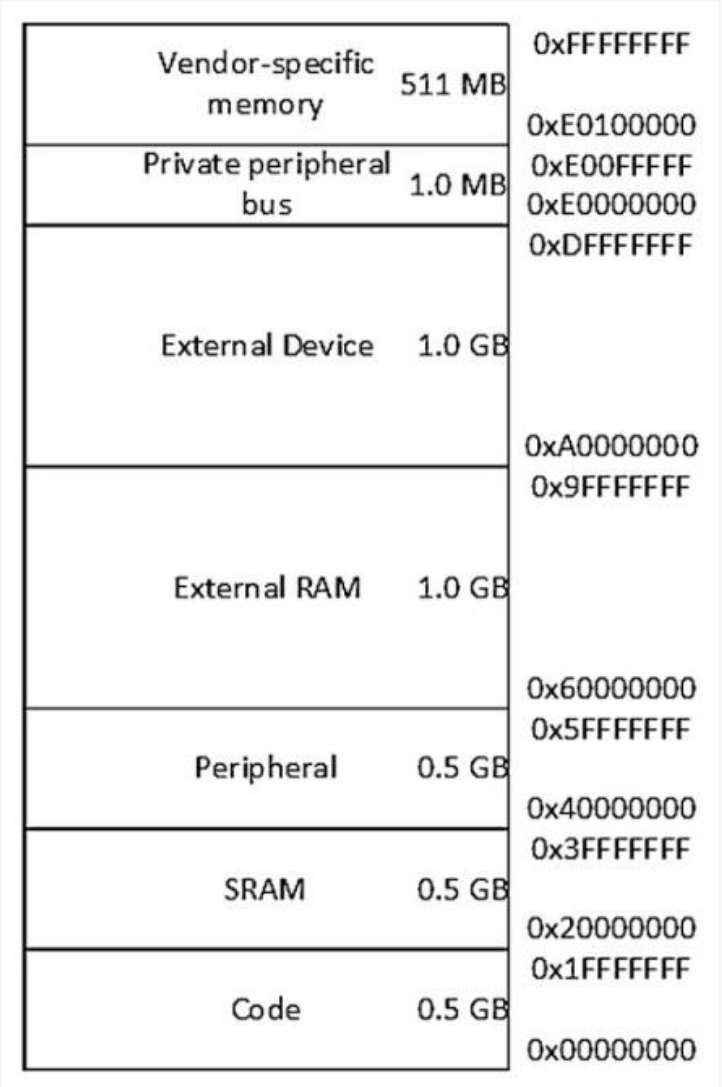


- When an instruction is executed within the CPU, the output is summarized as being negative, zero, or an overflow, carry, or saturation has occurred. These results are stored as condition flags (each being one bit) within the program status register (PSR).
- The negative (N) flag is set to logic level 1 when the result of an operation is negative. It is set to logic level 0 otherwise.
- The zero (Z) flag is set to logic level 1 when the result of an operation is zero. It is set to logic level 0 otherwise.
- The overflow (V) flag is set to logic level 1 when the result of an operation causes an overflow. It is set to logic level 0 otherwise.
- The carry (C) flag is set to logic level 1 when the result of an operation generates a carry. It is set to logic level 0 otherwise.
- The saturation (Q) flag is set to logic level 1 when the result of an operation causes saturation. It is set to logic level 0 otherwise.

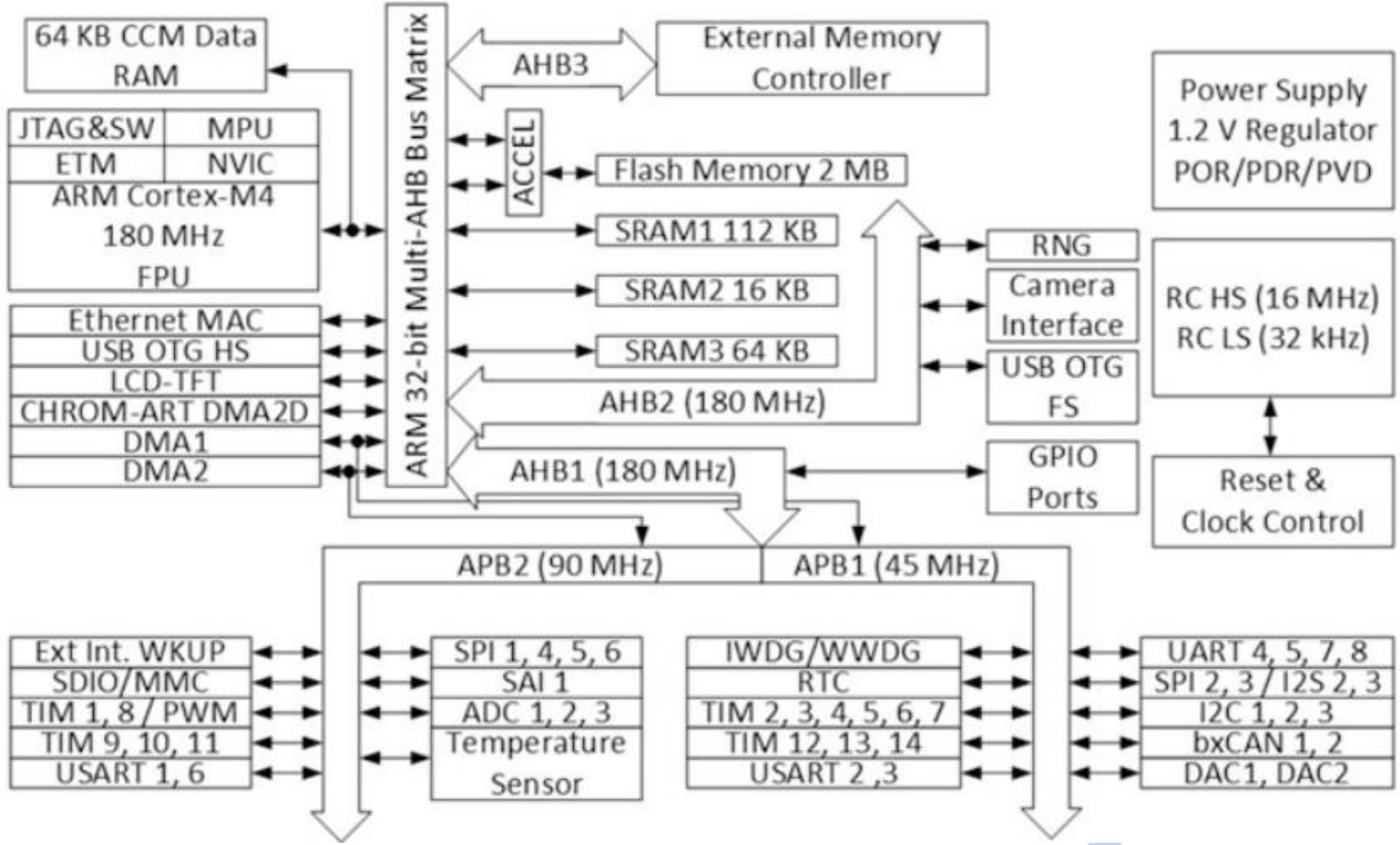
- There are two memory types in microcontrollers. There is random access memory (RAM) to store temporary data. Contents of RAM are lost when the power fed to it is turned off. There is also read-only memory (ROM) to store code or data permanently. Even if the power fed to it is turned off, contents of ROM are kept.
- RAM. In fact, it tells us that any randomly picked region within RAM can be accessed (reached) at the same time duration. Likewise, ROM may be misleading such that we only read its content. So, when and how was code or data written to it? Well, sometime in the operation, we should have written data on ROM.
- The STM32F4 microcontroller has two memory types as static RAM (SRAM) and flash. SRAM is a special memory type which does not need periodic refreshing as RAM. It also provides faster access to data. The STM32F4 microcontroller has 256 kB SRAM.
- The first 64 kB of SRAM is called core coupled memory (CCR) which provides fast code execution. There is also an extra 4 kB backup SRAM within the microcontroller which keeps data written to it even if the microcontroller goes into standby mode.

- The STM32F4 microcontroller has 2 MB flash memory. This memory is divided into two banks and each being composed of 12 sectors. These sectors are formed by 16 kB, 64 kB, or 128 kB memory regions. This structure allows the programmer to erase a specific sector, bank, or the overall flash memory.
- Arm® partitioned the memory address range for its microcontroller cores based on usage. Since the STM32F4 microcontroller is based on the Arm® Cortex™-M4 architecture, it has the same memory map.
- There is an extremely important concept to be mentioned here. The memory map of the STM32F4 microcontroller does not only represent SRAM and flash memory addresses. It also represents the interrupt and reset vector table, on-chip peripheral units, off-chip memory, off-chip peripherals, system components, and on- and off-chip debug components.
- The first 512 MB memory space is for code to be executed on the microcontroller. The next 512 MB memory space is used for on-chip SRAM. Code to be executed can also be stored here. However, this creates extra wait states and longer execution times. Therefore, running the code from this region is not recommended by Arm®.

- The next 512 MB memory space is used for all on-chip peripheral unit (GPIO, timers, ADC, and DAC) registers.
- All peripheral units are also treated as if they are separate memory locations. Hence, if the CPU tries to reach a peripheral unit, this is done in the same way as reaching a memory location.
- Although the user does not see this operation while writing code in C or C++ languages, it is important to keep this in mind.
- The next 2 GB memory space is used for external memory and peripheral units.
- The final 512 MB memory space is used for system peripheral units such as NVIC, memory protection unit (MPU), and SysTick (specific timer to be explored in Chap. 6). This memory space is also used for internal-external debug and vendor specific components



- There are also memory related modules within the STM32F4 microcontroller. The first one is the direct memory access (DMA) module. We can explain the importance of this module as follows. The CPU is used in most memory reaching operations. However, it may be busy for some cases.
- The DMA module allows reaching memory without interfering the CPU. Hence, the CPU can work on its own task. At the same time, reaching memory can be done in parallel by the DMA module.
- The STM32F4 microcontroller has DMA controllers.
- Besides these, STM32F4 has flexible memory controller (FMC) and chrom-art accelerator controller (DMA2D) modules.


64 KB CCM Data RAM
JTAG&SW
MPU
ETM
NVIC
ARM Cortex-M4
180 MHz
FPU
Ethernet MAC
USB OTG HS
LCD-TFT
CHROM-ART DMA2D
DMA1
DMA2
ARM 32-bit Multi-AHB Bus Matrix
AHB3
External Memory Controller
ACCEL
Flash Memory 2 MB
SRAM1 112 KB
SRAM2 16 KB
SRAM3 64 KB
AHB2 (180 MHz)
AHB1 (180 MHz)
RNG
Camera Interface
USB OTG FS
GPIO Ports
Power Supply
1.2 V Regulator
POR/PDR/PVD
RC HS (16 MHz)
RC LS (32 kHz)
Reset & Clock Control
APB2 (90 MHz)
APB1 (45 MHz)
Ext Int. WKUP
SDIO/MMC
TIM 1, 8 / PWM
TIM 9, 10, 11
USART 1, 6
SPI 1, 4, 5, 6
SAI 1
ADC 1, 2, 3
Temperature Sensor
IWDG/WWDG
RTC
TIM 2, 3, 4, 5, 6, 7
TIM 12, 13, 14
USART 2 ,3
UART 4, 5, 7, 8
SPI 2, 3 / I2S 2, 3
I2C 1, 2, 3
bxCAN 1, 2
DAC1, DAC2

Assembly Language

- When we write code in C or C++ language, in fact we are not writing in the format the microcontroller understands.
- The compiler converts our code to machine language and embeds it to code region of the microcontroller memory. Hence, it can be executed by the CPU.
- Since the code in machine language is stored in memory, it is in binary form. Therefore, understanding it is neither trivial nor feasible for us.
- Assembly language offers an option to write the code as close as possible to machine language but at the same time understandable by a human when inspected.
- In other words, we can write the code in assembly language in an editor which can be understood when inspected.
- Besides, assembly language allows us to reach low-level hardware of the microcontroller.
- Therefore, using it provides a good insight in understanding inner working principles of the microcontroller.

VisUAL2

```
ORR R1, R1, #16 ; sets bit 4 in R1
BICS R10, R10, 0x80000; clear bit 19 of R10, setting falgs NZ
```

OpCode	Function	Flags changed if S Suffix
AND	dest := op1 Bitwise AND op2	NZ
ORR	dest := op1 Bitwise OR op2	NZ
EOR	dest := op1 Bitwise exclusive OR op2	NZ
BIC	dest := op1 Bitwise AND NOT op2	NZ

```
MVN R1, R1   ; inverts bits in R1
MOVS R10, R3 ; load R10 with copy of value in R3, write flags `NZ`.
```

OpCode	Function	Flags changed if S suffix
MOV	dest := op2	NZ
MVN	dest := Bitwise invert op2	NZ

```
SUB R10, R5, R4 ; set R10 equal to R5 - R4
ADD R1, R1, #1  ; add 1 to R1
RSBS R1, R1, #0 ; negate R1 setting NZCV (R1 = 0 - R1)
```

OpCode	Function	Flags changed if S Suffix
ADD	dest := op1 + op2	NZCV
SUB	dest := op1 - op2	NZCV
RSB	dest := op2 - op1	NZCV
ADC	dest := op1 + op2 + C	NZCV
SBC	dest := op1 - op2 + C - 1	NZCV
RSC	dest := op2 - op1 + C - 1	NZCV

OpCode	Example	Notes
DCD	`DAT DCD d1, d2,...,dn`	Set next n data words to numeric values given by operands d1,...,dn (no #). Label `DAT` is usual but not compulsory
DCB	`DAT1 DCB d1, d2,...,dn`	Set next n data bytes to numeric values given by operands d1,...,dn (no #). n must be divisible by 4. Bytes are written in little-endian address order. Label `DAT1` is usual but not compulsory.
FILL	`DAT2 FILL 40`	`40` can be any literal numeric expression (without #). Fill next 40 words of data memory with 0 Label `DAT2` is usual but not compulsory.
ADR	`ADR Ra, LABEL`	Set Ra to `LABEL`. Same as `MOV Ra, #LABEL` but literal values close to PC are allowed. Useful to load data area label values.
EQU	`LAB1 EQU LABEL + 4`	`EQU` sets its label, in this case `LAB1`, equal to the given numeric expression. Forward references are allowed.

```
LSLS R0, R1, #4     ; same as MOVS R0, R1, LSL #4
ROR  R1, R1, #2     ; same as MOV R1, R1, ROR #2
RRX  R1, R2         ; same as MOV R1, R2, RRX
```

These are shorthand for the corresponding `MOV` with shifted *op2*.

OpCode	Function
LSR	Logical Shift Right (shift 0s in)
ASR	Arithmetic Shift Right (shift sign bit in)
LSL	Logical Shift Left
ROR	Rotate Right (shift RHS bit in)
RRX	Rotate Right eXtended - always 1 bit

```
CMP R0, R1 ; set `NZCV` flags on subtraction R0 - R1
TEQ R3, R4 ; set `NZ` flags on R3 XOR R4
```

OpCode	Function	Flags changed
CMP	set flags on op1 - op2	NZCV
CMN	set flags on op1 + op2	NZCV
TST	set flags on op1 Bitwise AND op2	NZ
TEQ	set flags on op1 Bitwise XOR op2	NZ

```
LDR R0, [R1,#8] ; load into R0 mem32[R1+8]
STRB R11, [R2,#3]! ; store LS byte R11 in mem8[R2+3], set R2 := R2+3
LDR R0, [R1],#8 ; load into R0 mem32[R1]. then set R1 := R1+8.
LDR R5, =DATA ; load 32 bit immediate value of symbol DATA into R5
```

See Address Modes for details of **EA** modes.

OpCode	Instruction	Function	Notes
LDR	`LDR Rd, EA`	Rd := mem32[EA]	1
LDRB	`LDRB Rd, EA`	Rd := mem8[EA]	2
STR	`STR Rs, EA`	mem32[EA] := Rs	1
STRB	`STRB Rs, EA`	mem8[EA] := Rs	3

LDR =	`LDR Rd, =Literal`
	`LDR Rd, =DATA+4`
	`LDR Rd, =1571`
		Rd := Literal	4

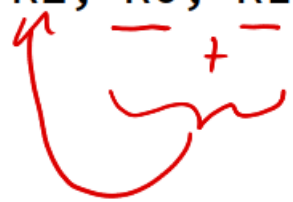
1.EA must be divisible by 4.

2.This loads one byte from memory into the LS 8 bits of Rd. The MS 24 bits are zeroed.

3.This writes the LS 8 bits of Rd into the specified memory byte. Other bytes in the same memory word are unaffected.

4.The literal can be any numeric expression of constants and symbols, and does not have a #. Unlike MOV there is no restriction on value. Technically this instruction is a pseudo-instruction, translated by assembler into an LDR with EA offset from PC, and a DCD that defines the the 32 bit constant. Programmers can use it exactly as MOV with literal op2, noting that it is slower than MOV.

```
MOV R0, #2 ; Put 2 into R0
MOV R1, #3 ; Put 3 into R1
ADD R2, R0, R1 ; R2 = R0 + R1
```



```
LDR R0, [R1] ; Load the memory at address R1 and put it into R0
```

Register to load into

Address

```
STR R0, [R1] ; Store the value of R0 into memory at address R1
```

Register to save

Address

Adding to a memory location

```
MOV R1, #0x1000 ; Put the value 0x1000 into R1
LDR R0, [R1] ; The part in brackets is an address (a pointer)
ADD R0, R0, #2 ; Do the addition
STR R0, [R1] ; Store the result back
```

Equivalent C code:

```
int* p = (int*)0x1000
*p += 2;
```

Append "`S`" to an ALU command to make it set the CPSR
`CMP` is like `SUBS`, but doesn't put the result anywhere

Append "`EQ`", "`NE`", "`GE`", etc. to make an operation conditional

C code

```
if (a >= 100) {
    b = 5;
}
```

Equivalent assembly

```
CMP R0, #100
MOVGE R1, #5
```

Practice

Write a program that computes the absolute value of a number stored in **R0**.

Absolute value

```
CMP R0, #0 ; Compare it to zero
RSBLE R0, R0, #0 ; If < or =, do R0 = 0 - R0
```

Challenge

Write a program that computes the absolute value of a value stored in memory. You can use these lines to define data in memory and load its address:

```
DATA DCD -130; or whatever value you want
MOV R1, #DATA
```

```
MOV R1, #DATA ; Put the address of DATA into R1
LDR R0, [R1] ; Load the value at that address into R0
CMP R0, #0 ; Compare it to zero
RSBLE R0, R0, #0 ; If < or =, do R0 = 0 - R0
STR R0, [R1] ; Store the result back
```

Branches

A branch lets you jump to another place in the program

```
        MOV     R1, #0x1000
START   LDR     R0, [R1]
        ADD     R0, R0, #1
        STR     R0, [R1]
        B       START
```

```
int* p = (int*)0x1000;
while(1){
    *p++;
}
```

Branches

Branching *conditionally* allows you to build if/while/for/case/etc.

```
        MOV     R1, #0x1000
START   LDR     R0, [R1]
        ADD     R0, R0, #1
        STR     R0, [R1]
        CMP     R0, #100
        BNE     START
```

```
int* p = (int*)0x1000;
while(*p != 100){
    *p++;
}
```

(not quite! C checks the condition at the top.)

Practice!

Write code to loop through the numbers 1-10, and compute their sum.

Example implementation in C:

```
int sum = 0;
for(int i = 1; i <= 10; i++){
    sum = sum + i;
}
```

Loops

A while loop can be written in the form:

```
TOP
    check if condition is false, and branch to BOTTOM if so
    [ body of loop ]
    unconditionally branch back to TOP
BOTTOM
```

```
1   data    DCD     0xFF, 0xEE, 0xDD
2
3
4           MOV     R0, #5
5           MOV     R1, #-5
6           MOV     R2, #018
7           MOV     R3, #0x7FFFFFFF
8
9           ADDS    R3, R3, #1
10
11          SUBS    R4, R0, R1
12
13          ADR     R0, 0x300
14
15          STR     R4, [R0]
16
17
18          LDR     R6, [R0]
19
20
```

```
1   value   DCD     44, 0x44
2
3           MOV     R0, #0x200
4
5           LDR     R1, [R0]
6
7           ADD     R0, R0, #4
8
9           LDR     R2, [R0]
10
11
12          ;       AND R3, R1, R2
13
14          ;       ORR R4, R1, R2
15          ;       EOR R5, R1, R2
16
17          MVN     R3, R2
18
19          BIC     R4, R1, R2
20
```

```
1   dat1    DCD     -5
2
3
4           ;       LDR R0, =dat1
5           MOV     R0, #dat1
6
7
8           LDR     R1, [R0]
9
10          CMP     R1, #0
11
12          RSBLT   R2, R1, #0
13
14
15          LSL     R3, R1, #1
16
17          LSR     R4, R2, #1
18
```